Computational Physics Laboratory report

Stefano Ge

Winter semester 2025

Contents

0 Introduction **2**

1 Error analysis **2**
- 1.0.1 Approximating exponentials . . . 2
- 1.2 Floating-point arithmetic and roundoff errors . . . 3
  - 1.2.1 Computing the Basel problem . . . 3
- 1.3 Error propagation and condition number . . . 5
  - 1.3.1 Computing statistical momenta . . . 5
  - 1.3.2 Condition number: study of a simple algorithm . . . 5

2 Linear systems **5**
- 2.1 Forward- and back-substitution . . . 5
- 2.2 LUP Decomposition . . . 6
- 2.3 Conditioning of linear systems . . . 7
- 2.4 Overdetermined linear systems . . . 8
  - 2.4.1 Least squares through normal equations . . . 8
  - 2.4.2 Fitting of Kepler's Law . . . 8
  - 2.4.3 Fitting of exp(sine) function . . . 9
- 2.5 QR decomposition . . . 11
  - 2.5.1 Least squares through QR . . . 11
  - 2.5.2 QR decomposition for eigenvalues and eigenvectors . . . 11

3 Interpolation **11**
- 3.1 Polynomial interpolation by solving the Vandermonde matrix linear system . . . 11
- 3.2 Lagrange-Waring interpolation . . . 12
  - 3.2.1 Evenly spaced nodes . . . 12
  - 3.2.2 Chebyshev nodes . . . 12
  - 3.2.3 Chebyshev vs evenly spaced nodes: error study . . . 13
  - 3.2.4 Polynomial interpolation over the real line with Chebyshev nodes . . . 13
  - 3.2.5 Chebyshev nodes interpolation of non-smooth functions . 13

4 Roots of nonlinear equations **14**
4.1 Bisection method 14
4.2 Newton method 15
4.2.1 Convergence rate study 15
4.2.2 Which root gets found by Newton's method? 17
4.2.3 Eros' true anomaly 17
4.3 Secant method 18
4.4 Inverse quadratic interpolation 18

5 Numerical integration **18**
5.1 Newton-Cotes formulas 18
5.1.1 Trapezoidal rule 18
5.1.2 Simpson's rule 18
5.2 Gauss-Legendre rule 18
5.2.1 Nodes and weights of Gauss-Legendre rule 18
5.2.2 Integrals with Gauss-Legendre rule 19
5.3 Double exponential rule 19
5.3.1 Comparison of double exponential and Gauss-Legendre rules 19
5.3.2 Improper integrals with double exponential rule 19

6 Ordinary differential equations **19**
6.1 Euler's method 19
6.1.1 First order application 19
6.1.2 Second order application 19
6.2 Runge-Kutta methods 19
6.2.1 Improved Euler's and Runge-Kutta fourth order methods 19
6.2.2 Non-linear equation application of Runge-Kutta 19
6.2.3 Predator-prey model 19

7 Schrödinger equation **19**
7.1 Time-dependant Schrödinger equation 19
7.1.1 Gaussian packet in an infinite well 19
7.1.2 Moving gaussian packet 19
7.1.3 Finite barrier scattering 19
7.1.4 Pöschl-Teller potential 19
7.2 Time-independant Schrödinger equation 19

0 Introduction

1 Error analysis

1.0.1 Approximating exponentials

Results

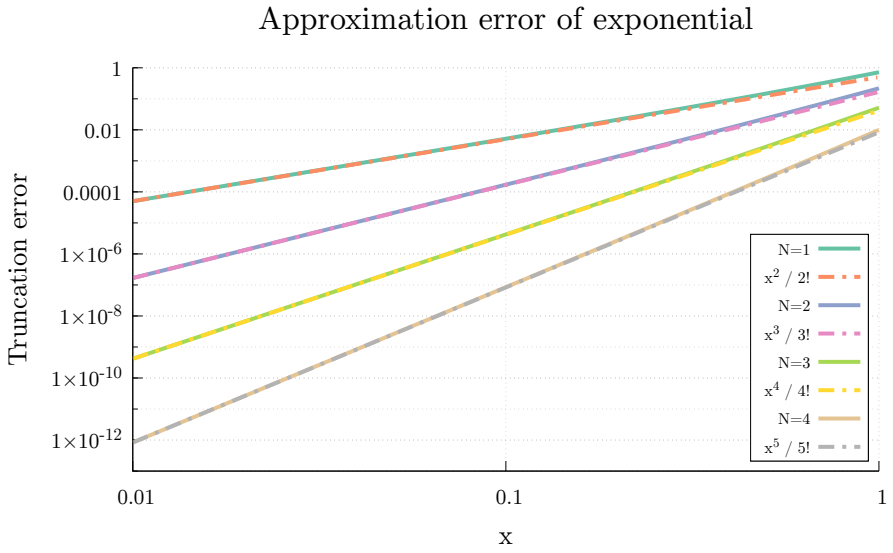


Figure 1: Truncation errors when approximating e^x through the truncated power series Σ_0^N. N is the degree of the polinomyal used for the approximation. The plot of $x^{N+1}/(N+1)$ has been added for reference.

Remarks

- Horner's method was used in polynomial evaluation, because it is faster and reduces roundoff errors. A naive implementation of a polynomial computation requires more operations ($O(N^2)$) and, for small values of x, adds together numbers of order 1 and x^N, while Horner's method is $O(N)$ and keeps the additions only between terms of order 1 and x.
- By Taylor's theorem, if $\hat{f}(x)$ is the power series of e^x truncated after the N-th term, the error is of order $O(x^{N+1})$ as x approaches 0. Computing an additional term of the series shows that:

$$|e^x - \hat{f}(x)| = \frac{x^{N+1}}{(N+1)!} + o(x^{N+1})$$

  It is then not surprising that the approximation error in Figure 1 is indistinguishable from $x^{N+1}/(N+1)$ for small values of x. For larger values, the contribution of higher order terms in the series becomes ever more important, and hence the divergence from $x^{N+1}/(N+1)$.

1.2 Floating-point arithmetic and roundoff errors

1.2.1 Computing the Basel problem

Results

Remarks

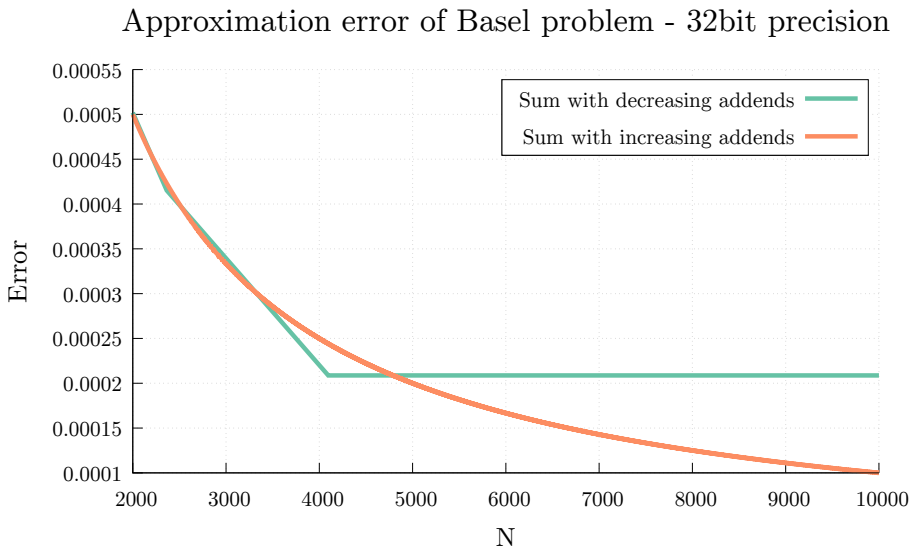


Figure 2: Approximation error when computing $\pi^2/6$ with the Basel problem truncated series $\Sigma_0^N \frac{1}{n^2}$, with single precision floating variables. Here N is the number of terms used when calculating the series.

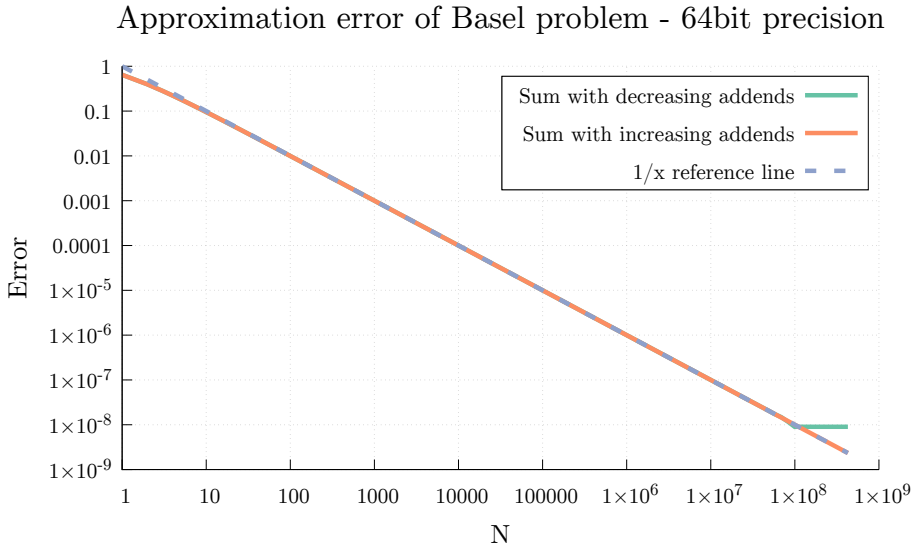


Figure 3: Same as above, but now using double precision floating variables. the plot of $1/x$ has been added for reference.

- The algorithm has been implemented with two mathematically equivalent forms, which, however, resulted in remarkably different numerical results. In the first the terms of the series are summed in increasing order of the summation index, i.e. decreasing of the value of the addends. Since small addends are summed to the accumulating partial sum, it gets to a point where the next addend is smaller than $\epsilon_{mach} \cdot sum \approx \epsilon_{mach}$, so the addition of it and subsequent values does not change the value of the partial sum.
  This can be seen in the error plots in Figure 2 and 3, since the error stops decreasing and remains constant after $1/\mathrm{N}^2 = \epsilon_{\mathrm{mach}}$, which is 2^{-24} for single precision ($\mathrm{N} = 4096$) and 2^{-53} for double precision ($\mathrm{N} \approx 9.5 \times 10^7$).

  In the second implementation the terms of the series are summed in decreasing order of the summation index, i.e. increasing the value of the addends. Therefore, the aforementioned effect is not seen, and the numerical results correspond to the mathematical expectation, in the range considered.

- The truncation error of this algorithm is asymptotically equivalent to k/N, with k real constant. This can be from the integral associated to $\Sigma_{n=0}^{\infty}$:

$$\int_N^{\infty} \frac{1}{n^2} = \frac{1}{N}$$

  therefore,

$$\Sigma_{n=N}^{\infty} \sim_{N\to\infty} \frac{1}{N}$$

  which is confirmed from the agreement between the computed values and the $1/x$ reference line in Figure 3.

1.3 Error propagation and condition number

1.3.1 Computing statistical momenta

1.3.2 Condition number: study of a simple algorithm

2 Linear systems

2.1 Forward- and back-substitution

Results

$$\begin{bmatrix} -2 & 0 & 0 \\ 1 & -1 & 0 \\ 3 & 2 & 1 \end{bmatrix} \mathbf{X} = \begin{bmatrix} -4 \\ 2 \\ 1 \end{bmatrix} \qquad \mathbf{X} = \begin{bmatrix} 2 \\ 0 \\ -5 \end{bmatrix}$$

$$\begin{bmatrix} 4 & 0 & 0 & 0 \\ 1 & -2 & 0 & 0 \\ -1 & 4 & 4 & 0 \\ 2 & -5 & 5 & 1 \end{bmatrix} \mathbf{X} = \begin{bmatrix} -4 \\ 1 \\ -3 \\ 5 \end{bmatrix} \qquad \mathbf{X} = \begin{bmatrix} -1 \\ -1 \\ 0 \\ 2 \end{bmatrix}$$

$$\begin{bmatrix} 3 & 1 & 0 \\ 0 & -1 & -2 \\ 0 & 0 & 3 \end{bmatrix} \mathbf{X} = \begin{bmatrix} 1 \\ 1 \\ 6 \end{bmatrix} \qquad \mathbf{X} = \begin{bmatrix} 2 \\ -5 \\ 2 \end{bmatrix}$$

$$\begin{bmatrix} 3 & 1 & 0 & 6 \\ 0 & -1 & -2 & 7 \\ 0 & 0 & 3 & 4 \\ 0 & 0 & 0 & 5 \end{bmatrix} \mathbf{X} = \begin{bmatrix} 4 \\ 1 \\ 1 \\ 5 \end{bmatrix} \qquad \mathbf{X} = \begin{bmatrix} -3.\overline{3} \\ 8 \\ -1 \\ 1 \end{bmatrix}$$

The results obtained through my routine were verified using LAPACK.

2.2 LUP Decomposition

Results

$$\begin{bmatrix} 2 & 3 & 4 \\ 4 & 5 & 10 \\ 4 & 8 & 2 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0.5 & 0.\overline{3} & 1 \end{bmatrix} \begin{bmatrix} 4 & 8 & 2 \\ 0 & -3 & 8 \\ 0 & 0 & 0.\overline{3} \end{bmatrix}$$

Determinant is 4.

$$\begin{bmatrix} 6 & -2 & -4 & 4 \\ 3 & -3 & -6 & 1 \\ -12 & 8 & 21 & -8 \\ -6 & 0 & -10 & 7 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0.5 & 1 & 0 & 0 \\ -0.25 & 0.25 & 1 & 0 \\ -0.5 & -0.5 & -0.857143 & 1 \end{bmatrix} \begin{bmatrix} -12 & 8 & 21 & -8 \\ 0 & -4 & -20.5 & 11 \\ 0 & 0 & 4.375 & -3.75 \\ 0 & 0 & 0 & 2.28571 \end{bmatrix}$$

Determinant is -480.

$$\begin{bmatrix} 1 & 4 & 5 & -5 \\ -1 & 0 & -1 & -5 \\ 1 & 3 & -1 & 2 \\ 1 & -1 & 5 & -1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0.8 & 1 & 0 \\ -1 & -0.2 & -0.\overline{6} & 1 \end{bmatrix} \begin{bmatrix} 1 & -1 & 5 & -1 \\ 0 & 5 & 0 & -4 \\ 0 & 0 & -6 & 6.2 \\ 0 & 0 & 0 & -2.\overline{6} \end{bmatrix}$$

Determinant is 80.

Matrix $\mathbf{A} = \mathbf{T}(3, -1)\mathbf{R}(\pi/5)\mathbf{T}(-3, 1)$:

$$A = \begin{bmatrix} 2.3 & 1.6 & -2.3 \\ -1.1 & 0.48 & 2.7 \\ 0.5 & 0.\overline{3} & -0.\overline{6} \end{bmatrix}$$

$$\mathbf{b} = \mathbf{Az} = \mathbf{A} \begin{bmatrix} 2 \\ 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 5.45 \\ 1.51 \\ 1.5 \end{bmatrix}$$

Factorization of $\mathbf{A}$ is:

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 0 \\ -0.471 & 1 & 0 \\ 0.217 & -0.00857 & 1 \end{bmatrix} \begin{bmatrix} 2.31 & 1.59 & -2.34 \\ 0 & 1.22 & 1.64 \\ 0 & 0 & 0.354 \end{bmatrix}$$

2.3 Conditioning of linear systems

Results In the case study proposed, first we construct numerically:

$$\mathbf{b} = \mathbf{A}\mathbf{x} = \begin{bmatrix} -\epsilon & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Where ϵ is a real parameter. We then apply the LU decomposition (without pivoting) to solve for x given $\mathbf{A}$ and the $\mathbf{b}$ just computed.

For $\epsilon = 10^{-12}$ we find:

$$\mathbf{A} = \mathbf{L}\mathbf{U} = \begin{bmatrix} 1 & 0 \\ -1\text{e}{-12} & 1 \end{bmatrix} \begin{bmatrix} -\text{e}{-12} & 1 \\ 0 & 1\text{e}12 \end{bmatrix}$$

$$\mathbf{x}_{\text{computed}} - \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -2.2\text{e}{-5} \\ 0 \end{bmatrix}$$

As a test, we find correctly:

$$\mathbf{A} - \mathbf{L}\mathbf{U} = \mathbf{0}$$

However, if we set $\epsilon = 10^{-12}$, we find:

$$\mathbf{A} = \mathbf{L}\mathbf{U} = \begin{bmatrix} 1 & 0 \\ -1\text{e}{-20} & 1 \end{bmatrix} \begin{bmatrix} -\text{e}{-20} & 1 \\ 0 & 1\text{e}20 \end{bmatrix}$$

$$\mathbf{x}_{\text{computed}} - \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 0 \end{bmatrix}$$

$$\mathbf{A} - \mathbf{L}\mathbf{U} = \begin{bmatrix} 0 & 0 \\ 0 & -1 \end{bmatrix}$$

Remarks

- The exact LU-factorization of $\mathbf{A}$ is:

  $$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ -\epsilon^{-1} & 1 \end{bmatrix} \begin{bmatrix} -\epsilon & 1 \\ 0 & \epsilon^{-1} - 1 \end{bmatrix}$$

  If $\epsilon = 10^{-20} \ll \epsilon_{\text{mach}}$, in the bottom-right entry of $\mathbf{U}$ only ϵ^{-1} gets stored, since the relative variation of adding -1 to ϵ^{-1} is less than ϵ_{mach}. But when $\mathbf{L}$ and $\mathbf{U}$ get multiplied together, in the addition resulting into the bottom-right entry of $\mathbf{A}$ two ϵ^{-1} terms cancels out, and the exact result is the -1 that is lost in numerical LU-decomposition.

- Exact $\mathbf{b}$ is:

  $$\begin{bmatrix} 1 - \epsilon \\ 0 \end{bmatrix}$$

  but ϵ does not get registered if it is lower than ϵ_{mach}. Because the condition number of the matrices $\mathbf{L}$ and $\mathbf{U}$ is very high (ϵ^{-2} in ∞-norm), the tiny variation of $\mathbf{b}_{\text{calc}}$ over $\mathbf{b}_{\text{exact}}$ is allowed to propagate tremendously and results in the wrong result seen before.

  Doing the calculations shows that the error on $\mathbf{U}$ as no impact on $\mathbf{b}_{\text{calc}}$, and that the final error on $\mathbf{b}_{\text{calc}}$ is due entirely on the ill-conditioning of the linear systems $\mathbf{L}\mathbf{z} = \mathbf{b}$ and $\mathbf{U}\mathbf{x} = \mathbf{z}$.

- By contrast, the condition number of the starting matrix $\mathbf{A}$ is only ≈ 4. Therefore, in principle it should be possible to solve the system for $\mathbf{x}$ even if $\mathbf{b}$ is slightly nudged: the problem is the stability of the algorithm adopted.

  LU-decomposition without row-pivoting is much less numerically stable than LUP-decomposition with row-pivoting where the maximum value entry in a column is selected as the pivot.

  Indeed, if we perform LU-decomposition on the following matrix, obtained by permuting the two rows of matrix $\mathbf{A}$, as prescribed by row-pivoting, we find the correct results:

$$\mathbf{PA} = \begin{bmatrix} 1 & -1 \\ -\epsilon & 1 \end{bmatrix} = \mathbf{LU} = \begin{bmatrix} 1 & 0 \\ -1\text{e}{-20} & 1 \end{bmatrix} \begin{bmatrix} ta1 & -1 \\ 0 & 1 \end{bmatrix}$$

$$\mathbf{x}_{\text{computed}} - \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$\mathbf{A} - \mathbf{LU} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

2.4 Overdetermined linear systems

2.4.1 Least squares through normal equations

Results After implementing a routine for finding a the vector $\mathbf{X}$ that best fits an overdetermined linear system, we tested it on the following linear system:

$$\begin{bmatrix} 2 & -1 \\ 0 & 1 \\ -2 & 2 \end{bmatrix} \mathbf{X} = \begin{bmatrix} 1 \\ -5 \\ 6 \end{bmatrix}$$

and found:

$$\mathbf{X} = \begin{bmatrix} -2 \\ 1 \end{bmatrix}$$

2.4.2 Fitting of Kepler's Law

We want to extrapolate Kepler's third law from experimental data through the algorithm just introduced. First we linearize the following relationship by taking the logarithm at both sides:

$$\mathrm{T} = \mathrm{CD}^{\alpha} \implies \log(\mathrm{T}) = \log(\mathrm{C}) + \alpha \log(\mathrm{D})$$

where T is the orbital periods of the planets, D is their average distance from the Sun and alpha and C are two costants to be determined through the method of least squares. This is equivalent to finding x_1 and x_2 that minimize:

$$\log(\mathbf{T}) = x_1\mathbf{1} + x_2 \log(\mathbf{D}) = \begin{bmatrix} \mathbf{1} & \log(\mathbf{D}) \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

where quantities in bold are vectors, leading to an overdetermined linear system.

Results We therefore find that the values that fit the aforementioned system are:

$$\alpha = 1.49865 \qquad C = 0.201522$$

where alpha is consistent with Kepler's law, which predicts $\alpha = 1.5$.

The graph containing the fitted curve and experimental data can be found in Figure 4

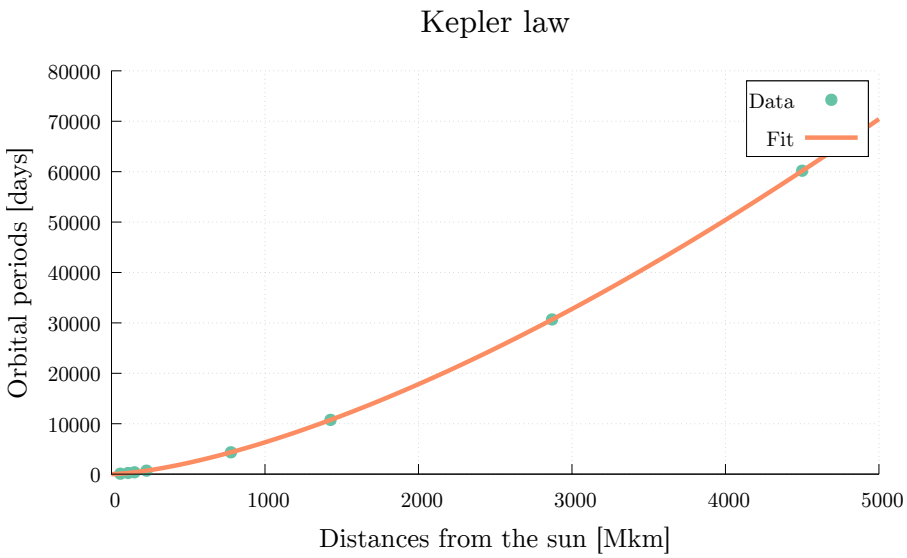


Figure 4: Orbital peirods vs distances from the Sun of the planets.

2.4.3 Fitting of exp(sine) function

Results We want to approximate the function $\exp(\sin(t-1))$, first through a Taylor-like polynomial:

$$y(t) = c_1 + c_2 t + ... + c_7 t^6$$

and then through a periodic Fourier-like approximation:

$$y(t) = d_1 + d_2 \cos(t) + d_3 \sin(t) + d_4 \cos(2t) + d_5 \sin(5t)$$

Results are shown in Figure 5 and in Figure 6.

Remarks

- Not surpringly, the function is better approximated by the first terms of the Fourier basis than by a sixth degree polynomial. The Fourier basis is an orthonormal complete set of $L^2([0, 2\pi])$, therefore it is guaranteed to be the best approximation in L^2 norm, among expansions on sets spanned by it. Moreover, sines and cosines are themselves periodic and are thus well suited for approximating a periodic function like $\exp^{\sin(t-1)}$, producing a nice approximation over the whole domain $\mathbb{R}$. On the other hand, monic

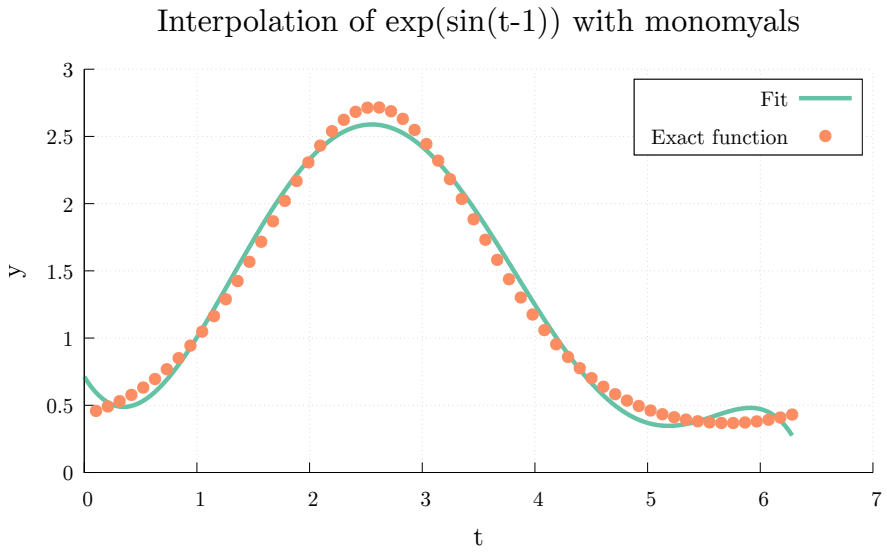


Figure 5: Least square fit of exp(sin($t-1$) with 6th degree polynomial

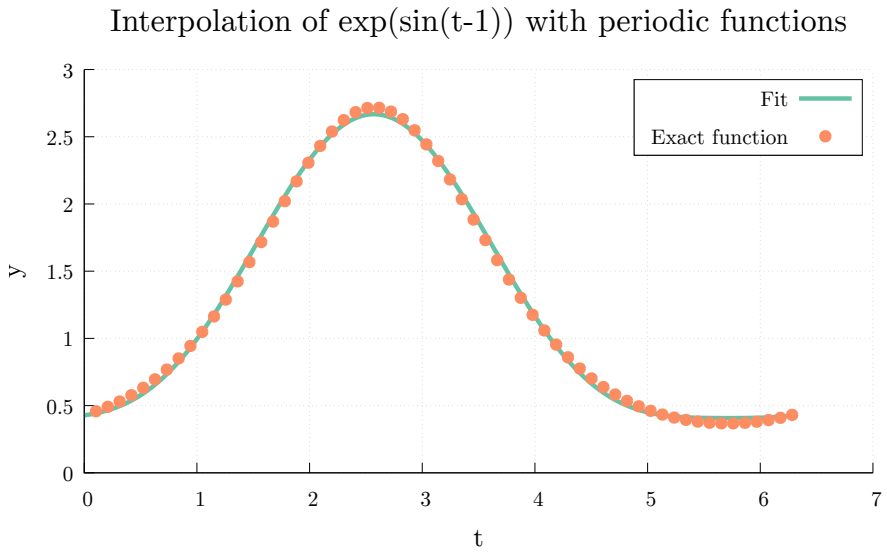


Figure 6: Least square fit of exp(sin($t-1$) with Fourier periodic functions

monomyals, though complete, are neither an orthonormal set of $L^2([0, 2\pi])$ nor are well suited for approximating periodic functions, as they do blow up at infinity and do not approximate well outside of the points employed in the fit.

2.5 QR decomposition

In this section we implement algorithms to find the QR decomposition of any matrices. First the modified Gram-Schmidt procedure (MGS).

Results We test the modified Gram-Schmidt procedure on the following matrices:

$$\begin{bmatrix} 2 & -1 \\ 0 & 1 \\ -2 & 2 \end{bmatrix} = \begin{bmatrix} 0.707 & 0.408 \\ 0.000 & 0.816 \\ -0.707 & 0.408 \end{bmatrix} \begin{bmatrix} 2.828 & -2.121 \\ 0 & 1.225 \end{bmatrix}$$

$$\begin{bmatrix} 2 & 3 & 4 \\ 4 & 5 & 10 \\ 4 & 8 & 2 \end{bmatrix} = \begin{bmatrix} 0.333 & -0.104 & 0.937 \\ 0.667 & -0.677 & -0.312 \\ 0.667 & 0.729 & -0.156 \end{bmatrix} \begin{bmatrix} 6.000 & 9.667 & 9.333 \\ 0 & 2.134 & -5.726 \\ 0 & 0 & 0.312 \end{bmatrix}$$

$$\begin{bmatrix} 2 & -1 & 4 \\ 0 & 3 & 5 \\ 7 & 2 & -6 \\ 1 & -4 & 0 \end{bmatrix} = \begin{bmatrix} 0.272 & -0.241 & 0.733 \\ 0.000 & 0.559 & 0.617 \\ 0.953 & 0.179 & -0.233 \\ 0.136 & -0.773 & 0.163 \end{bmatrix} \begin{bmatrix} 7.348 & 1.089 & -4.627 \\ 0 & 5.368 & 0.752 \\ 0 & 0 & 7.418 \end{bmatrix}$$

Computing the product $\mathbf{A} = \mathbf{QR}$ gives us the matrices we have decomposed, confirming the correctness of the decomposition. Also we computed $\mathbf{Q}^\intercal\mathbf{Q}$ and we confirmed that the computed $\mathbf{Q}$'s are indeed ONC matrices.

2.5.1 Least squares through QR

We implemented QR algorithm to solve least-squares problem, and applied it to the problem of fitting the function $\exp^{\sin(4t)}$ with a 14th degree polynomial.

Results

2.5.2 QR decomposition for eigenvalues and eigenvectors

3 Interpolation

3.1 Polynomial interpolation by solving the Vandermonde matrix linear system

Interpolating polynomials in the power basis by constructing the Vandermonde matrix and solving the related linear system is numerically unstable. Tha matrix thus constructed is likely to be ill-conditioned, due to the fact that its entries are the nth powers of x, and can become quickly extremely large or small as the system scales in size.

In our case study, we considered equidistant point in the interval [-1,1] generated by the function $\cos(kx)$.

In Figure AGGIUNGI REFERENCE we can see that the condition number scales exponentially with the number of points considered in the fit, which is also the size of the linear system.

AGGIUNGI FIGURA

In our study we always considered $4n + 1$ equidistant points, including the extrema of the interval -1 and 1. As a consequence, the middle point considered is always (0,1), since $\cos(0) = 1$. Therefore, we must have:

$$P_n(0) = a_0 + a_1 \cdot 0 + ... + a_n \cdot 0 = a_0 = 1$$

CERCA DI CAPIRE COME MOSTRARE I VALORI

If we were operating in exact arithmetic, this would produce a value of a_0 always equal to 1, since it would lead to a linear system where a row of the Vandermonde matrix (the one corresponding to point x=0) which is equivalent to equation $a_0 = 1$. However, the LUP decomposition algorithm contaminates several times the aforementioned row of the linear system, relying later on cancellation of possibly large values to recover the information of $a_0 = 1$. Therefore, in finite-precision arithmetic the value of a_0 can differ from 1, as we can see ... . This is closely related to the ill-conditioning of the Vandermonde matrix.

3.2 Lagrange-Waring interpolation

3.2.1 Evenly spaced nodes

Results In Figure ADD 4 GRAPHS AND REFERENCES we show the polynomials resulting from the Lagrange-Waring interpolation of the the functions $\log(x)$, $tanh(x)$, $cosh(x)$ and $|x|$, using equally spaced nodes.

The weights of the points were evaluated through the analytic formula for equally spaced weights. Some lines of code have been added to the code in order to show that the special formula agrees with the generic one.

Remarks

- The Lagrange-Waring interpolation implementation has been organised around a struct, `BarFit`, which stores points, weights, etc.

  This OOP-style design is flexible, and allows reusage of the weights already calculated for other functions.

3.2.2 Chebyshev nodes

Results We have tested our routine for Lagrange interpolation using Chebyshev nodes of the second kind by fitting the function $cosh(sin(x))$ using 40 points in the interval $[0, 2\pi]$.

The interpolating polynomial, together with the nodes, can be visualized in ADD REFERENCE

In the testing program we also included a a comparison between the weights calculated through the analytical formula for Chebyshev nodes and the generic nodes formula. It confirmed that the weights are the same aside from roundoff errors.

3.2.3 Chebyshev vs evenly spaced nodes: error study

We study the convergence of Lagrange intrpolation wit equidistant an Chebyshev points for the functions $1/(25x^2 + 1)$, $\tanh(5x + 2)$, $\cosh(\sin(x))$ and $\sin(\cosh(x))$ over the inerval [-1:1].

The results are shown in ADD REFERENCS AND FIGURES

Remarks

- Interpolation theory has proven that, for analytical functions bounded in the interpolation interval, $||f - p||_\infty \leq CK^n$ for some constants C and K, i.e. that Lagrange's interpolation with Chebyshev nodes converges at least exponentially. In our plot we see this exact pattern. We can even estimate the constant K:

f(x)	K
$1/(25x^2 + 1)$	1.22
$\tanh(5x + 2)$	1.39
$\cosh(\sin x)$	5.70
$\sin(\cosh x)$	4.81

- When we use equidistant points as nodes convergence is not guaranteed for every analytical and bounded functions. Instead, as the number of nodes grows, $||f - p||_\infty$ can grow exponentially, that is Runge's phenomenon. We observe this phenomenon in our plot.

3.2.4 Polynomial interpolation over the real line with Chebyshev nodes

By means of a change of variable we now interpolate the function $1/(z^2-2z+2)$ over an unbounded interval [-∞, +∞]. We used Chebyshev nodes of the first kind to avoid evaluation at x = $\pm\infty$.

As change of variable we used the function $\phi(x) = \frac{2x}{1-x^2}$ which maps the interval [-1, 1] to the real line.

Results are shown in ADD REFERENCE AND FIGURE

3.2.5 Chebyshev nodes interpolation of non-smooth functions

Polynomial interpolants on Chebyshev nodes are guaranteed to converge exponentially to the reference function if the latter is analytic, as the number of nodes approaches infinity. If it is not, convergence might not be guaranteed or it might be slower.

We can observe this slowdown by studying the convergence for the family of functions $|x|^m$. These functions are not smooth on the interval [-1, 1]. They can be differentiated $m - 1$ times, but not m times, since $|m|$ is not differentiable in $x = 0$. Hence they are not analytic. The results are shown in ADD REFERENCE AND FIGURES

Remarks

- As previously said, $|x|^m$ is not analytical. However, their derivatives are continuously differentiable up to $f^{(m-2)}$, thus absolutely continuous. $f^{(m-1)} = C|x|$, which is Lipschitz continuous and as such it is absolutely continuous. On the other hand:

$$f^{(m)}(x) = \begin{cases} +C, & x > 0 \\ -C, & x < 0 \end{cases}$$

  for some C>0. This function is monotonic, hence it is of bounded variation. This proves a theorem that states that if the above hypotheses are verified (absolute continuity up to $m-1$-th derivative and bounded variation of m-th derivative), then the Chebyshev polynomial interpolant converges to the function in sup-norm as $\mathcal{O}(n^{-m})$, as the number of nodes n approaches infinity.

- To compare this prediction with our results, we interpolated the sup-norm error using algorithm from ADD REFERENCES By choosing $Y = \log_{10}(||f-p||_\infty) = \log_{10}(c) - K\log_{10}(n)$ and $X = \log_{10}(n)$ we interpolated the relation:

$$Y = C - K\,X$$

  where -K is the exponent in the asymptotic behaviour $\mathcal{O}(n^{-m})$ of $||f-p||_\infty$. The fitting lines can are visualised in ADD REFERENCE as dashed lines. Our results are reported in the following table and are in accordance to the prediction above:

m	K
1	1.04
3	3.13
5	5.15
7	7.26
9	9.41
11	11.7

4 Roots of nonlinear equations

4.1 Bisection method

In order to test our routine for rootfinding with the bisection method, we searched for the roots of the function $x^3 - 7x^2 + 14x - 6$ in the interval [0, 1] and found $x = 0.5858$. The plot (which we omitted from the report, but can be visualized during code execution) of the function and the found root confirms that it is the only root in the interval.

Given an iterative rootfinding algorithm, if we define $\epsilon_n = x_n - r$, where r is the root that we are going to find and x_n is the root estimate at step n, then if

$$\lim_{n\to+\infty} \left| \frac{\epsilon_{n+1}}{\epsilon_n^q} \right| = C \tag{1}$$

for $q \geq 1$ and $0 < C < \infty$, we say that the algorithm converges to the root with order of convergence q and asymptotic constant C.

It is very easy to see that for the bisection algorithm $q = 1$ and $C = \frac{1}{2}$. Under this assumption, we get the asymptotic relation

$$|\epsilon_{n+1}| \approx |\frac{1}{2}\epsilon_n|$$

which is the asymptotic recurrence relation of a geometric sequence $\epsilon_n = K(\frac{1}{2})^n$.

Defining $a_n = -\log_{10}(|\epsilon_n|)$ we get:

$$a_n = K' + n\log_{10}(2)$$

In ADD FIGURE AND REFERENCE we plotted the a_n as a function of n. The curve is a line, confirming that the order of convergence is 1. Fitting the curve with a line through least squares yields a slope of 1.992, which implies an asymptotic error constant $C = 0.5018$, in accordance with bisection method's convergence theory.

As a termination condition of the loop in the algorithm we check if:

$$|x_1 - x_2| <= \text{tol} + \epsilon_{\text{mach}} \max(|a|, |b|)$$

Where $|a|$ and $|b|$ are the boundaries of the starting interval and x_1, x_2 are the boundaries of the interval at current step.

`tol` can be specified by the caller or instead be 10ϵmach if the caller inserts the macro `DEF_TOL` as parameter.

$\epsilon_{\text{mach}} \max(|a|, |b|)$ is added because it is approximately the distance between the largest of the two starting boundaries (in absolute value). The algorithm is not always able to bisect the algorithm further when it is of the order of the quantity above, so it is a safety check.

4.2 Newton method

4.2.1 Convergence rate study

We seek to check the correctness of our routine and study the convergence rate of Newton's method by applying it to the functions $f_1(x) = x^2 - \exp(-x)$, $f_2(x) = \tan(x) - 2x$ and $f_3(x) = \exp(x + 1) - x - 2$.

By calculating $a_n = -log_{10}|x_n - r|$ we can calculate how many digits of precision we are gaining at each step, and estimate order of convergence and asymptotic error constant.

Results In ADD FIGURES AND REFERENCES we plotted the functions and the roots found.

In table 1 we quote the roots found and the numerical estimates for the values of q and C.

q is estimated as the asymptotic value of a_{n+1}/a_n, since it holds $a_{n+1} \sim qa_n$.

C is estimated as

$$|\epsilon_{n+1}/\epsilon_n^q|$$

where we use for q the value that we calculated analytically (see Remarks).

f(x)	Root	≈q	≈C
$x^2 - \exp(-x)$	0.703	2.08	0.40
$\tan(x) - 2x$	0.000	3.07	0.67
	1.166	1.92	3.38
$\exp(x+1) - x - 2$	-1.000	1.06	0.50

Table 1:

Remarks

- If a root r of a function $f(x)$ satisfies $f'(r) \neq 0$ and $f''(r) \neq 0$, then Newton's method's convergence order is quadratic for that root and the asymptotic error constant is:

$$C = \frac{1}{2}\left|\frac{f''(r)}{f'(r)}\right|$$

  That is the case for the root of f_1 and for root $x = 1.166$ of f_2. In both cases the estimated convergence order is close to 2 and the estimated asymptotic error constant agrees with the one calculated analitically by 4+ significant digits. Hence, we do not include separately the analitical values in the report and we refer the reader to the program `newton_method_error_study.c`.

- If the root is not simple, i.e. $f'(r) = 0$, but $f''(x) \neq 0$, then it can be shown that convergence is linear with asymptotic error constant $C = 0.5$. Root the root of f_3 satisfies this condition. In our analysis we see that most ratios a_{n+1}/a_n agree wih 0.5, except for the last few values.

- For root $r = 0$ of f_3 the estimated order of convergence is about 3. Seeing that for this function $f''(r) = 0$, this prompted us to check with a small computation whether $f''(r) = 0$ implies order of convergence 3. We start from the relation:

$$\epsilon_{k+1} = \epsilon_k - \frac{f(r+\epsilon_k)}{f'(r+\epsilon_k)}$$

  which is a rewrite of Newton's algorithm. Expanding with Taylor's:

$$\epsilon_{k+1} = \epsilon_k - \frac{f(r) + f'(r)\epsilon_k + \frac{1}{2}\epsilon_k^2 f''(r) + \frac{1}{6}\epsilon_k^3 f'''(r) + O(\epsilon_k^4)}{f'(r) + f''(r)\epsilon_k + \frac{1}{2}\epsilon_k^2 f'''(r) + O(\epsilon_k^3)} =$$

$$= \epsilon_k - \frac{f'(r)\epsilon_k + \frac{1}{6}\epsilon_k^3 f'''(r) + O(\epsilon_k^4)}{f'(r) + \frac{1}{2}\epsilon_k^2 f'''(r) + O(\epsilon_k^3)} =$$

  Collecting $f'(r)$:

$$= \epsilon_k \left[1 - \left(1 + \frac{1}{6}\epsilon_k^2 \frac{f'''(r)}{f'(r)}\right)\left(1 + \frac{1}{2}\epsilon_k^2 \frac{f'''(r)}{f'(r)}\right)^{-1}\right] + O(\epsilon_k^4) =$$

  By doing some simple algebra:

$$= \frac{1}{3}\epsilon_k^3 \frac{f'''(r)}{f'(r)} + O(\epsilon_k^4)$$

i.e. cubic convergence, with asymptotic error constant $C = \frac{1}{3}f'''(r)/f'(r)$. By executing `newton_method_error_study`, the reader can see that the numerically estimated and the analitically calculated one agree by 4 significant digits, after only 6 steps have been taken by Newton's method.

4.2.2 Which root gets found by Newton's method?

Newton method, in general, is not globally convergent. Moreover, if two or more roots are present, if convergent there is no guarantee that the root that gets found is the one "closest" to the starting value for the algorithm.

In this paragraph we study the relationship between the starting value and the root that gets found by the method. We use as case study the function $f(x) = x^{-2} + \sin(x)$ and as starting values for the algorithm $x = 1, 2, ..., 7$.

In ADD FIGURE AND REFERENCES we have plotted $f(x)$, along with the starting values and the related roots that get found.

We see that the initial guesses 2 and 5 produce return roots that are not the closest to the starting points. In order to understand what is going on, we have drawn the tangent line to the graph at each of those two initial guesses. Newton method approximates the function to that line at the first iteration. We can see from the graph the tangent lines drawn intersect the x axis "far away" from the starting guess, since at both points the function has a slope small in absolute value.

If we include bracketing in our algorithm, then we can be sure that the root that gets found will be inside of the interval from which we started.

ADD FIGURE AND REFERENCE shows the root found inside the arbitrality chosen intervals [0.5, 2], [2, 5], [5, 8] and [8, 10].

4.2.3 Eros' true anomaly

As an application of Newton method, we seek to evaluate the *true anomaly* $\theta(t)$ of asteroid Eros, defined as:

$$\tan\frac{\theta}{2} = \sqrt{\frac{1+\epsilon}{1-\epsilon}}\tan\frac{\psi}{2}$$

For that, we need to evaluate first the *eccentric anomaly* $\psi(t)$ which can be found by solving the equaton:

$$f(\psi) = \psi - \epsilon\sin\psi - \frac{2\pi t}{\tau} = 0 \tag{2}$$

For asteroid Eros, we have *orbial period* $\tau = 1.7610$ and *elliptical eccentricity* $\epsilon = 0.2230$.

Derivative of equation 2 with respect to ψ is:

$$f'(\psi) = 1 - \epsilon\cos\psi$$

Since $\epsilon < 1$, $f'(\psi) > 0$ for any ψ, the function is monotonic and we need not worry about finding multiple roots. Also, by plotting function 2 we see that it is reasonably well-behaved, and it might be even reasonable to expect global convergence. Nonetheless, we can also simply observe that for ψ such that $f(\psi) = 0$:

$$\left|\psi - \frac{2\pi t}{\tau}\right| = |\epsilon \sin \psi| \leq \epsilon$$

that is, if we safeguard Newton method with bracketing and we take as starting interval $[\frac{2\pi t}{\tau} - \epsilon, \frac{2\pi t}{\tau} + \epsilon]$ then we are guaranteed to find the root.

4.3 Secant method

4.4 Inverse quadratic interpolation

5 Numerical integration

5.1 Newton-Cotes formulas

5.1.1 Trapezoidal rule

5.1.2 Simpson's rule

5.2 Gauss-Legendre rule

5.2.1 Nodes and weights of Gauss-Legendre rule

Remarks

5.2.2 Integrals with Gauss-Legendre rule

5.3 Double exponential rule

5.3.1 Comparison of double exponential and Gauss-Legendre rules

5.3.2 Improper integrals with double exponential rule

6 Ordinary differential equations

6.1 Euler's method

6.1.1 First order application

6.1.2 Second order application

6.2 Runge-Kutta methods

6.2.1 Improved Euler's and Runge-Kutta fourth order methods

6.2.2 Non-linear equation application of Runge-Kutta

6.2.3 Predator-prey model

7 Schrödinger equation

7.1 Time-dependant Schrödinger equation

7.1.1 Gaussian packet in an infinite well

7.1.2 Moving gaussian packet

7.1.3 Finite barrier scattering

7.1.4 Pöschl-Teller potential

7.2 Time-independant Schrödinger equation